TD 4 : Mutex et section critique

Objectifs pédagogiques :

- **atomic, mutex**
- **sections critiques, interblocage**

1.1 Mise en place

On rappelle la classe `Compte` obtenue à la fin du TD 3.

```
class Compte {
      mutable mutex m;
      int solde;
public :
      Compte(int solde=0):solde(solde) {}
      void crediter (int val) {
            lock_guard g(m);
            solde+=val ;
      }
      bool debiter (int val) {
            lock_guard g(m);
            bool doit = solde >= val;
            if (doit) {
                  solde-=val ;
            }
            return doit;
      }
      int getSolde() const {
            lock_guard g(m);
            return solde;
      }
      // NB : vector exige Copyable, mais mutex ne l'est pas...
      Compte(const Compte & other) {
            lock_guard g(other.m);
            solde = other.solde;
      }
};
```

1.2 Transaction.

On considère à présent une Banque, possédant en attribut un ensemble de K comptes (un `vector<Compte>`) initialement avec un solde de $SOLDEINITIAL$ chacun. Elle est munie d'une opération `void transfert(size_t idDebit, size_t idCredit, int val)` qui essaie de débiter le compte d'indice *idDebit* de *val*, et si c'est un succès, crédite le compte *idCredit* du même montant *val*.

transferts.cpp

```
class Banque {
  using comptes_t = std::vector<Compte>;
  comptes_t comptes;
public :
  Banque (size_t ncomptes, size_t solde) : comptes (ncomptes, Compte(solde)){
  }
  void transfert(size_t deb, size_t cred, unsigned int val) {
    Compte & debiteur = comptes[deb];
    Compte & crediteur = comptes[cred];
    if (debiteur.debiter(val)) {
      crediteur.crediter(val);
    }
  }
  size_t size() const {
    return comptes.size();
  }
};
```

Question 1. Ecrivez un programme, qui crée N threads de transfert, qui bouclent un grand nombre de fois sur le comportement suivant :

- Choisir i et j deux indices de comptes aléatoires, et un montant aléatoire m (entre 70 et 100 dans le corrigé).
- Essayer de transférer le montant m de i à j.

Question 2. Les protections actuelles sur le Compte suffisent-elles à éviter les *data races* lors des transferts ?

On estime que découpler les débits des crédits est une faute, on souhaite au contraire que la mise à jour des deux comptes concernés soit atomique : soit le transfert a lieu et les deux comptes sont mis à jour (simultanément du point de vue d'un observateur), soit il n'a pas lieu. A aucun moment un observateur ne doit pouvoir voir un des comptes débités et l'autre pas encore crédité.

Question 3. Pour permettre à la Banque de verrouiller les comptes pendant un transfert, ajoutez à `Compte` les méthodes `void lock() const` et `void unlock() const`, qui délèguent leurs appels au mutex du compte.

Question 4. Proposez une stratégie de synchronisation pour permettre ce comportement transactionnel au niveau de `transfert`.

Question 5. Le programme se bloque immédiatement, même avec un seul thread qui fait des transferts. Pourquoi ?

Question 6. Après correction du problème précédent à l'aide de `recursive_mutex`, on introduit plusieurs thread faisant des transferts, mais de nouveau on observe parfois un interblocage, le programme entier se fige. Donnez un entrelacement de deux transferts qui mène à cet interblocage. Corrigez le problème en imposant un ordre d'acquisition des verrous sur les indices des comptes.

1.3 Comptabilité

On souhaite ajouter au programme des thread qui vérifient la comptabilité. On se donne une nouvelle opération `comptabiliser` dans la classe `Banque` :

BFL.cpp

```
1   bool comptabiliser (int attendu) const {
2         int bilan = 0;
3         int id = 0;
4         for (const auto & compte : comptes) {
5               if (compte.getSolde() < 0) {
6                     cout << "Compte " << id << " en négatif : " << compte.getSolde() << endl;
7               }
8               bilan += compte.getSolde();
9               id++;
10        }
11        if (bilan != attendu) {
12              cout << "Bilan comptable faux : attendu " << attendu << " obtenu : " << bilan << endl;
13        }
14        return bilan == attendu;
15  }
```

Cette méthode itère sur les comptes en sommant leur solde, et vérifie que la somme vaut bien $SOLDEINITIAL * K$. Sinon elle lève une alerte sur la console et rend false.

Question 7. Un thread comptable qui tourne en concurrence avec les threads de transfert sera-t-il satisfait avec les synchronisations actuelles ? Donnez un entrelacement qui produit un bilan incorrect.

Question 8. Ajoutez un mutex dans la banque, et les synchronisations utiles pour que le thread comptable obtienne toujours les bons résultats.

Question 9. Avec un seul mutex dans la banque, la concurrence entre les thread de transfert n'est plus possible. De fait les mutex définis dans Compte ne servent plus à rien dans ce scénario. Proposez une autre approche qui réutilise les locks de Compte plutôt que de n'avoir qu'un seul lock.

Indice : il faut juste empêcher les threads de transfert d'accéder aux comptes que le comptable a déjà vus, et comptés dans son `bilan`. Respectez le même ordre d'acquisition des verrous que les transferts pour ne pas réintroduire d'interblocage.

1.4 Une file bloquante

On souhaite construire classe `pr::BoundedBlockingQueue<T>` qui est une file bloquante FIFO munie d'une capacité fixe. Son API est la suivante :

- `BoundedBlockingQueue(size_t capacity)` : constructeur, la capacité est fixée à la construction.
- `void push(const T& value)` : ajoute un élément à la fin de la file. Si la file est pleine, le thread appelant est bloqué jusqu'à ce qu'une place se libère.
- `T pop()` : enlève et rend l'élément en tête de file. Si la file est vide, le thread appelant est bloqué jusqu'à ce qu'un élément soit disponible.

On fournit une version séquentielle basée sur `std::deque<T>`. Elle n'est ni protégée contre les accès concurrents, ni bloquante : elle suppose que l'appelant respecte les préconditions indiquées. La capacité est strictement positive.

Version séquentielle à compléter

```
namespace pr {
template <typename T>
class BoundedBlockingQueue {
   std::deque<T> queue_;
   size_t capacity_;
public:
   explicit BoundedBlockingQueue(size_t capacity)
      : capacity_(capacity) {}

   // Precondition : la file n'est pas pleine.
   void push(const T& value) {
      queue_.push_back(value);
   }

   // Precondition : la file n'est pas vide.
   T pop() {
      T value = queue_.front();
      queue_.pop_front();
      return value;
   }
};
}
```

Une classe de synchronisation : le pattern while/notify_all. On ne contrôle pas combien de clients accèdent à l'objet. On commence par rendre la classe MT-safe : un membre `mutable std::mutex m` protège son état, et chaque méthode qui accède à cet état prend le verrou, de préférence avec `lock_guard` ou `unique_lock`. Les fonctions privées appelées sous ce verrou n'ont pas à le reprendre.

Pour ajouter un comportement bloquant, on ajoute un membre `std::condition_variable cond` et on utilise le schéma suivant :

```
{
   std::unique_lock l(m);
   cond.wait(l, [&]() { return this->ok(); });
   // Equivalent : while (!this->ok()) cond.wait(l);
   // La precondition est vraie et le mutex est acquis.
   this->modify();
} // Fin de section critique : liberation du mutex.
cond.notify_all(); // A chaque modification utile.
```

`wait` libère le mutex pendant l'attente et le reprend avant de retester la précondition. Une notification invite les threads en attente à la vérifier à nouveau ; elle ne garantit pas qu'elle soit vraie pour chacun d'eux. On notifie après une modification susceptible de permettre à un autre appel de progresser. **Toujours commencer par cette stratégie : une condition, un test répété de la précondition et notify_all.**

Question 10. Complétez la classe pour réaliser le contrat de file bornée bloquante : protégez son état avec un mutex, attendez qu'une place ou un élément soit disponible, puis notifiez après la modification. Utilisez une seule condition et `notify_all`, en suivant le schéma ci-dessus. La classe doit supporter plusieurs appels concurrents à `push` et `pop`. Pourquoi ne peut-on pas attendre que la file change en conservant le mutex verrouillé ?